API SECURITY RISK ANALYSIS REPORT

Public API Security Assessment — JSONPlaceholder

Public API Security Assessment

Prepared for:	Future Interns Cyber Security Internship Program
Prepared by:	Mohammad Moinuddin
Role:	Cyber Security Intern — Track CS
CIN ID:	FIT/FEB26/CS6460
Task:	Task 3 — API Security Risk Analysis
Date:	March 18, 2026
Classification:	Confidential
GitHub Repo:	FUTURE_CS_03

1. Executive Summary

This report presents the findings of a read-only **API Security Risk Analysis** conducted on **JSONPlaceholder** (https://jsonplaceholder.typicode.com), a publicly available REST API used for testing and prototyping. The assessment was performed as part of the **Future Interns Cyber Security Internship Program 2026** (Task 3), by intern **Mohammad Moinuddin** (CIN: FIT/FEB26/CS6460).

Six security issues were identified across the API, including three HIGH severity findings related to the complete absence of authentication, insecure direct object references (IDOR), and excessive data exposure. These issues would pose serious risks in any real-world production API.

Total Findings	HIGH	MEDIUM	LOW	Scope
6 issues	3 findings	2 findings	1 finding	Read-only GET requests

2. Scope & Methodology

2.1 Target API

- API Name: JSONPlaceholder
- Base URL: https://jsonplaceholder.typicode.com
- API Type: Public REST API — test/demo environment

2.2 Endpoints Tested

Endpoint	Method	Purpose
/users	GET	Retrieve all user records
/users/1	GET	Retrieve specific user by ID (IDOR test)
/users/999	GET	Invalid ID — error handling test
/comments	GET	Retrieve all comments — data exposure test
/posts	GET	Retrieve all posts — pagination test

2.3 Tools Used

- Postman v11 — API request testing, response and header inspection
- Browser DevTools — supplementary header analysis
- OWASP API Security Top 10 (2023) — assessment framework

2.4 Ethical Boundaries

All testing was conducted using read-only GET requests only. No authentication bypass, exploitation, injection, brute force, or denial-of-service testing was performed. JSONPlaceholder is a public demo API specifically designed for safe testing and learning.

3. Findings Summary

#	Finding	Category	Severity	OWASP
1	No Authentication on Endpoints	Broken Object Level Auth	HIGH	API1:2023
2	IDOR — Access Any User by ID	Broken Object Level Auth	HIGH	API1:2023
3	Excessive Data Exposure (/comments)	Excessive Data Exposure	HIGH	API3:2023
4	Missing Critical Security Headers	Security Misconfiguration	MEDIUM	API8:2023
5	No Pagination — Mass Data Dump	Unrestricted Resource Use	MEDIUM	API4:2023
6	Technology Stack Disclosure	Security Misconfiguration	LOW	API8:2023

4. Detailed Findings

Finding 1 — No Authentication on All Endpoints

Severity	HIGH
OWASP	API1:2023 — Broken Object Level Authorization
Endpoints	GET /users, /comments, /posts, /users/{id}
Evidence	200 OK returned with full user data — no token, API key, or login required

Description

Every endpoint on this API is fully accessible without any authentication. No API key, Bearer token, session cookie, or any other credential is required. Any person on the internet can retrieve the full user dataset simply by knowing the URL. The screenshot below confirms a **200 OK** response returning complete user records including names, emails, addresses, phone numbers, and GPS coordinates — with zero authentication.

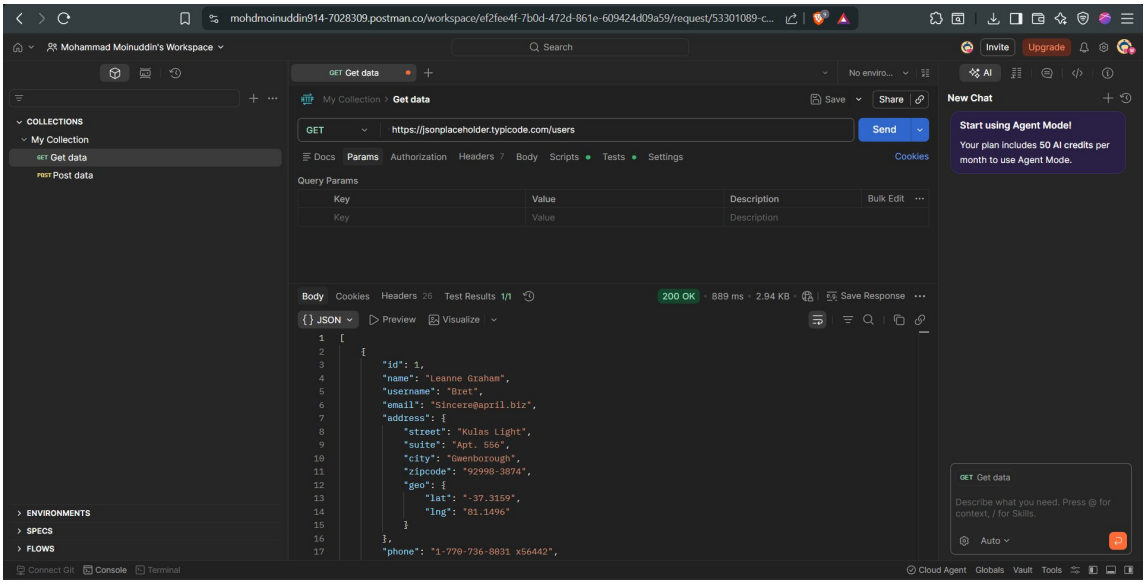


Figure 1: GET /users returns full user data with 200 OK — no authentication required

Business Impact

In a real production system, this would constitute a catastrophic data breach. All customer records would be publicly accessible, violating GDPR, CCPA, and most data protection laws. Fines under GDPR can reach up to €20 million or 4% of global annual turnover.

Remediation

- Implement OAuth 2.0 Bearer token or API key authentication on all endpoints
- Return 401 Unauthorized for all unauthenticated requests
- Use an API gateway (e.g. AWS API Gateway, Kong) to enforce auth at the network layer
- Apply role-based access control (RBAC) to restrict data access by user role

Finding 2 — Insecure Direct Object Reference (IDOR)

Severity	HIGH
OWASP	API1:2023 — Broken Object Level Authorization
Endpoint	GET /users/{id}
Evidence	Changing the ID integer (1, 2, 3...) returns any user's full record without any access check

Description

The API allows any caller to retrieve any user record by simply incrementing the numeric ID in the URL path. There is no check to verify whether the requester is authorised to access that specific record. An attacker could enumerate all user IDs sequentially and extract the entire user database in seconds using an automated script.

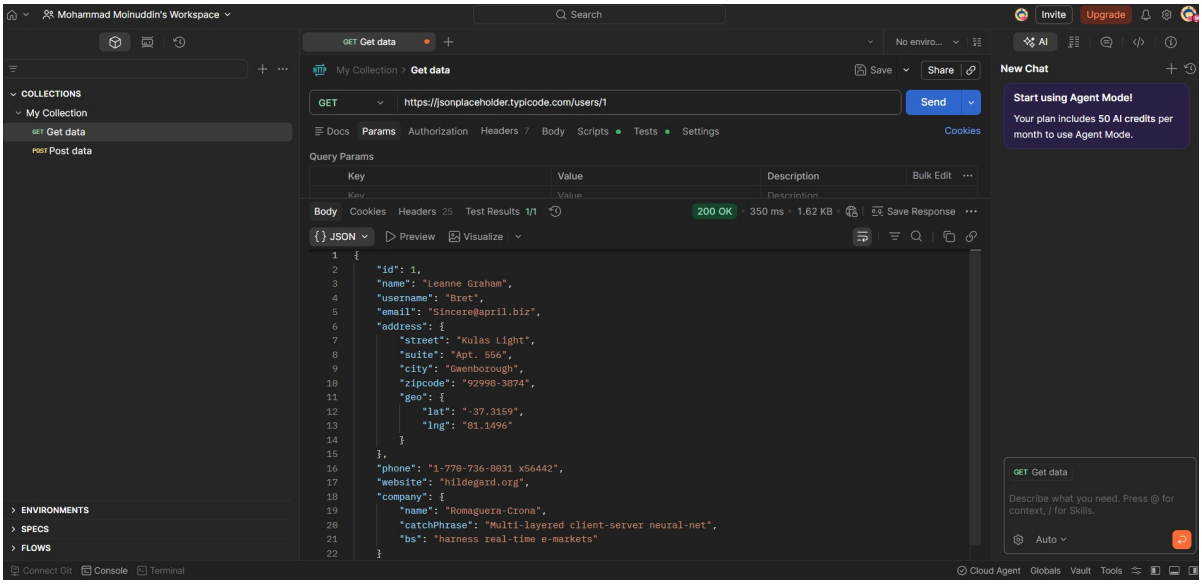


Figure 2: GET /users/1 returns full user record — any user accessible by changing the ID number

Business Impact

IDOR is consistently ranked among the most exploited API vulnerabilities. A complete user database can be harvested with a simple loop script. This represents a full data breach scenario.

Remediation

- Validate server-side that the authenticated user owns or has permission to access the requested resource
- Replace sequential integer IDs with random UUIDs (e.g. /users/a3f9b2c1-... instead of /users/1)
- Implement object-level authorisation checks on every API request — never rely on obscurity

Finding 3 — Excessive Data Exposure in /comments

Severity	HIGH
OWASP	API3:2023 — Broken Object Property Level Authorization
Endpoint	GET /comments
Evidence	Response: 155.17 KB — email address exposed in every comment record, no auth required

Description

The /comments endpoint returns the full comment objects including each commenter's email address. Email addresses are Personally Identifiable Information (PII) and should never be exposed in a public-facing API endpoint. The response returned 500 comment records (155 KB) — all without authentication. Multiple emails were visible: Eliseo@gardner.biz, Jayne_Kuhic@sydney.com, Nikita@garfield.biz, and hundreds more.

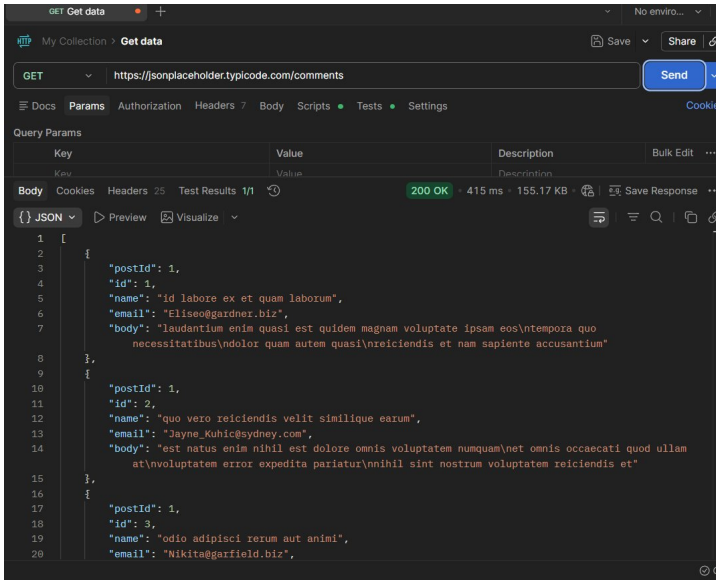


Figure 3: GET /comments exposes email addresses in all 500 records — 155 KB of unauthenticated data

Business Impact

Exposed emails can be harvested for phishing campaigns, spam, and credential stuffing attacks. This directly violates GDPR Article 5 (data minimisation). The API is returning far more data than the client needs.

Remediation

- Apply response filtering — return only the fields the client needs (e.g. name and body only for comments)
- Remove email fields from all public-facing comment response objects
- Implement a DTO (Data Transfer Object) serialisation layer to control field exposure per endpoint

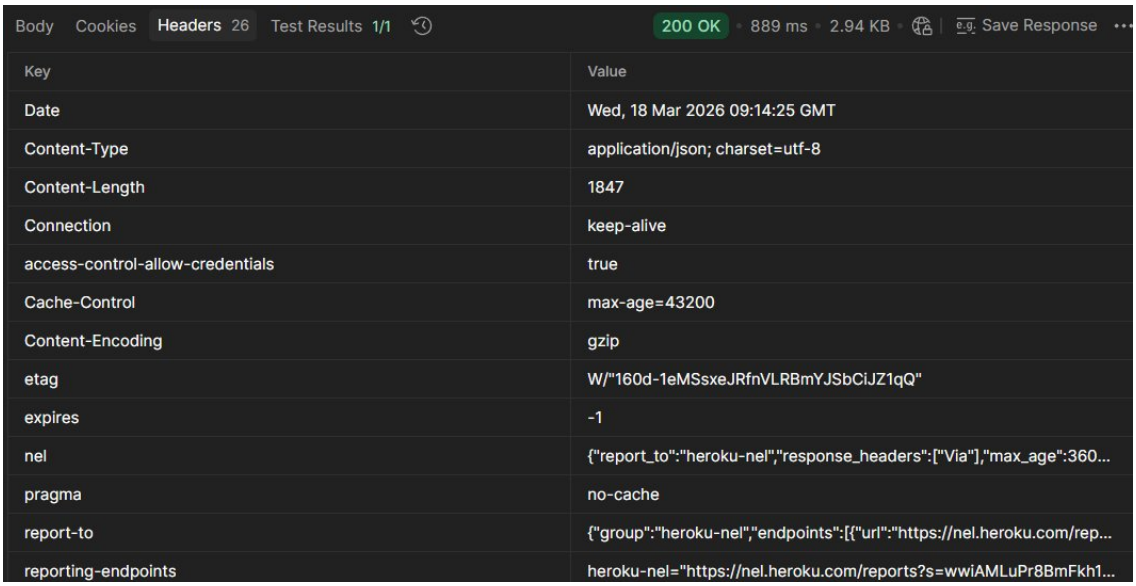
Finding 4 — Missing Critical Security Headers

Severity	MEDIUM
OWASP	API8:2023 — Security Misconfiguration
Evidence	Response headers inspected in Postman — 26 headers returned, critical security headers absent

Description

Inspection of the API response headers revealed that several critical security headers are absent. While some headers are present (x-content-type-options: nosniff, x-ratelimit headers), the following key headers are missing:

Missing Header	Purpose	Risk if Absent
Content-Security-Policy	Prevents XSS injection attacks	Cross-site scripting possible
X-Frame-Options	Prevents clickjacking / UI redressing	Iframe embedding attacks
Strict-Transport-Security	Forces HTTPS connections only	Man-in-the-middle attacks
Permissions-Policy	Restricts browser feature access	Browser feature abuse



Key	Value
Date	Wed, 18 Mar 2026 09:14:25 GMT
Content-Type	application/json; charset=utf-8
Content-Length	1847
Connection	keep-alive
access-control-allow-credentials	true
Cache-Control	max-age=43200
Content-Encoding	gzip
etag	W/"160d-1eMSsxeJRfnVLRBmYJSbCiJZ1qQ"
expires	-1
nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":360...
pragma	no-cache
report-to	{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/rep...
reporting-endpoints	heroku-nel="https://nel.heroku.com/reports?s=wwiAMLuPr8BmFkh1...

Figure 4a: Response headers — Part 1 (missing security headers visible)

Server	cloudflare
vary	Origin, Accept-Encoding
via	2.0 heroku-router
x-content-type-options	nosniff
x-powered-by	Express
x-ratelimit-limit	1000
x-ratelimit-remaining	999
x-ratelimit-reset	1773415395
Age	6661
Accept-Ranges	bytes
cf-cache-status	HIT
CF-RAY	9de320867a16398c-IAD
alt-svc	h3=":443"; ma=86400

Figure 4b: Response headers — Part 2 (x-powered-by: Express discloses technology stack)

Remediation

- Add Content-Security-Policy: default-src 'self' to all responses
- Add X-Frame-Options: DENY to prevent iframe embedding
- Add Strict-Transport-Security: max-age=31536000; includeSubDomains
- Configure headers at the web server or API gateway level (Nginx, Cloudflare, etc.)

Finding 5 — No Pagination (Mass Data Dump)

Severity	MEDIUM
OWASP	API4:2023 — Unrestricted Resource Consumption
Endpoint	GET /posts
Evidence	100 records returned in single response — 28 KB, no limit or pagination parameters

Description

The /posts endpoint returns all 100 records in a single response with no pagination, filtering, or maximum limit controls. In a production system with thousands of records, this would cause significant performance degradation and could be abused to trigger large data dumps repeatedly as a form of denial-of-service or data harvesting attack.

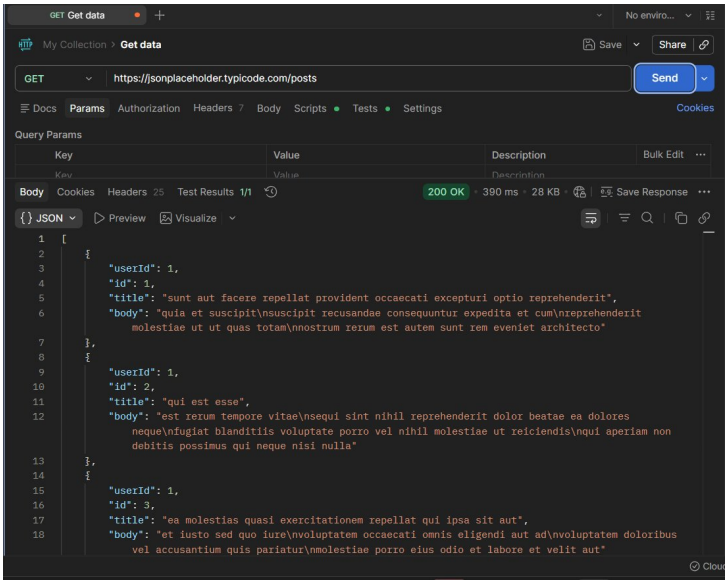


Figure 5: GET /posts returns all 100 records (28 KB) in a single response with no pagination

Remediation

- Implement cursor-based or offset pagination (e.g. GET /posts?page=1&limit;=20)
- Set and enforce a maximum page size limit — never return more than 100 records per request
- Return pagination metadata in every list response (totalCount, nextPage, prevPage)

Finding 6 — Technology Stack Disclosure

Severity	LOW
OWASP	API8:2023 — Security Misconfiguration
Evidence	Response header x-powered-by: Express reveals Node.js/Express.js framework

Description

The response header **x-powered-by: Express** reveals that the API is built on the Express.js framework running on Node.js. While low severity on its own, this provides attackers with valuable reconnaissance. If a

known vulnerability exists in the specific Express version, attackers know exactly which exploits to attempt.

Remediation

- Remove x-powered-by header in Express with: app.disable('x-powered-by')
- Audit all response headers and strip any that disclose internal technology details

Observation — Proper Error Handling (Positive Finding)

Severity	N/A — Good Practice
Endpoint	GET /users/999
Evidence	404 Not Found with empty body {} — no stack trace or internal details leaked

The API correctly returns a **404 Not Found** with an empty JSON body when a non-existent resource is requested. No internal error messages, stack traces, or server details are disclosed. This is a positive security practice worth noting.

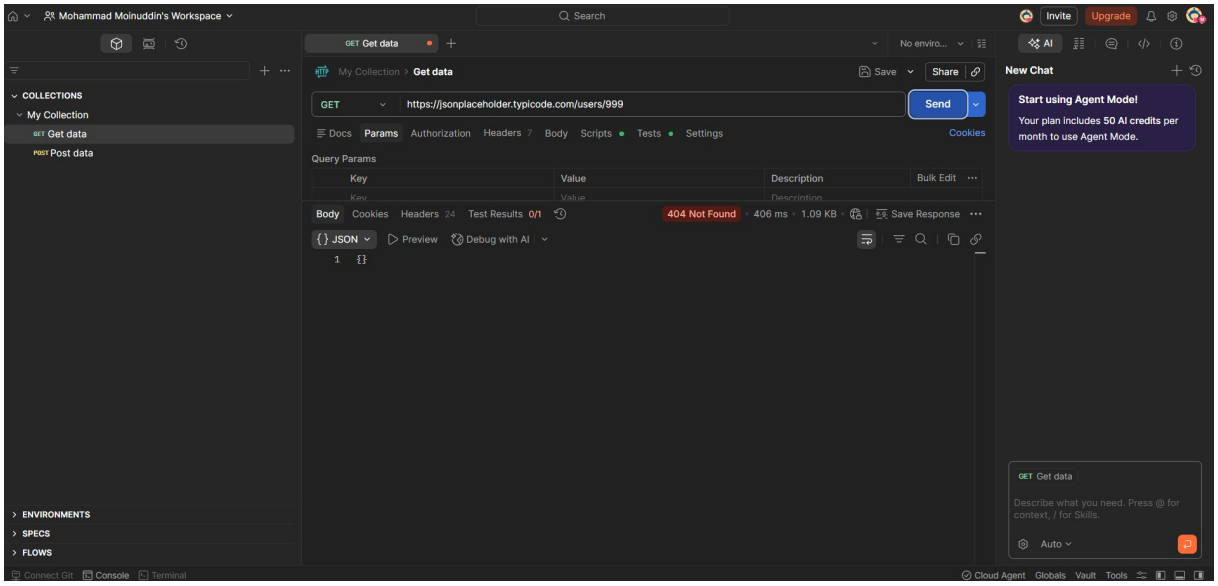


Figure 6: GET /users/999 returns clean 404 — no internal details leaked

5. Risk Matrix

#	Finding	Likelihood	Impact	Severity	Priority
1	No Authentication	High	Critical	**HIGH**	Fix Immediately
2	IDOR Vulnerability	High	Critical	**HIGH**	Fix Immediately
3	Excessive Data Exposure	High	High	**HIGH**	Fix Immediately
4	Missing Security Headers	Medium	Medium	**MEDIUM**	Fix Within 30 Days
5	No Pagination	Medium	Medium	**MEDIUM**	Fix Within 30 Days
6	Stack Disclosure	Low	Low	**LOW**	Fix Within 90 Days

6. Remediation Roadmap

Timeline	Actions Required	Findings
Immediate (0–7 days)	Implement authentication (OAuth 2.0 / API keys). Add object-level authorisation. Remove emails from comment responses.	1, 2, 3
Short-term (8–30 days)	Add all missing security headers (CSP, X-Frame-Options, HSTS). Implement pagination with maximum limits.	4, 5
Medium-term (31–90 days)	Remove x-powered-by header. Conduct full security review after fixes to verify remediation effectiveness.	6

7. Conclusion

This API Security Risk Analysis identified **six security vulnerabilities** in the JSONPlaceholder public test API. While this is an intentionally open demo platform, the vulnerabilities found represent patterns that are extremely common in real-world production APIs.

The three HIGH severity findings — absent authentication, IDOR, and excessive data exposure — represent the most critical risks and would constitute a data breach in any production environment. All identified issues are straightforward to remediate using well-established industry practices such as OAuth 2.0, UUID-based identifiers, response DTOs, and standard security headers.

All findings align with the **OWASP API Security Top 10 (2023)**, the industry-standard framework for API security assessment. Implementing the recommendations in this report would significantly improve the security posture of any API following similar patterns.

8. Ethical Statement

This assessment was conducted strictly within the following ethical boundaries:

- Only public-facing, read-only GET requests were made at all times
- No authentication bypass, exploitation, or injection attacks were attempted
- No denial-of-service or brute force testing was performed
- The target (JSONPlaceholder) is a public demo API designed specifically for testing
- All findings are based solely on passive observation of API responses and headers

---

This report was submitted as part of the **Future Interns Cyber Security Internship Program 2026**

Intern: Mohammad Moinuddin | CIN: FIT/FEB26/CS6460 | GitHub: FUTURE_CS_03

— End of Report —